

01_matrix_mult.c — Visual Studio Code

01_matrix_mult.c

```

1  /* Program 1: Matrix Multiplication */
2  #include <stdio.h>
3
4  int main() {
5      int a[10][10], b[10][10], c[10][10];
6      int r1, c1, r2, c2;
7
8      printf("Enter rows and columns of first matrix: ");
9      scanf("%d %d", &r1, &c1);
10     printf("Enter rows and columns of second matrix: ");
11     scanf("%d %d", &r2, &c2);
12
13     if (c1 != r2) {
14         printf("Matrix multiplication not possible (columns of A must equal rows of B)\n");
15         return 0;
16     }
17
18     printf("Enter elements of first matrix (%dx%d):\n", r1, c1);
19     for (int i = 0; i < r1; i++)
20         for (int j = 0; j < c1; j++)
21             scanf("%d", &a[i][j]);
22
23     printf("Enter elements of second matrix (%dx%d):\n", r2, c2);
24     for (int i = 0; i < r2; i++)
25         for (int j = 0; j < c2; j++)
26             scanf("%d", &b[i][j]);
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 01_matrix_mult.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./01_matrix_mult
Enter rows and columns of first matrix: 2 2
Enter rows and columns of second matrix: 2 2
Enter elements of first matrix (2x2):
1 2
3 4
Enter elements of second matrix (2x2):
5 6
7 8

Resultant Matrix:
19 22

```

02_odd_even.c — Visual Studio Code

02_odd_even.c

```

1  /* Program 2: Find Odd or Even number from a given set of numbers */
2  #include <stdio.h>
3
4  int main() {
5      int n;
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      int arr[100];
10     printf("Enter %d numbers: ", n);
11     for (int i = 0; i < n; i++)
12         scanf("%d", &arr[i]);
13
14     printf("\nResult:\n");
15     for (int i = 0; i < n; i++) {
16         if (arr[i] % 2 == 0)
17             printf("%d is Even\n", arr[i]);
18         else
19             printf("%d is Odd\n", arr[i]);
20     }
21     return 0;
22 }
23

```

PROBLEMS OUTPUT TERMINAL

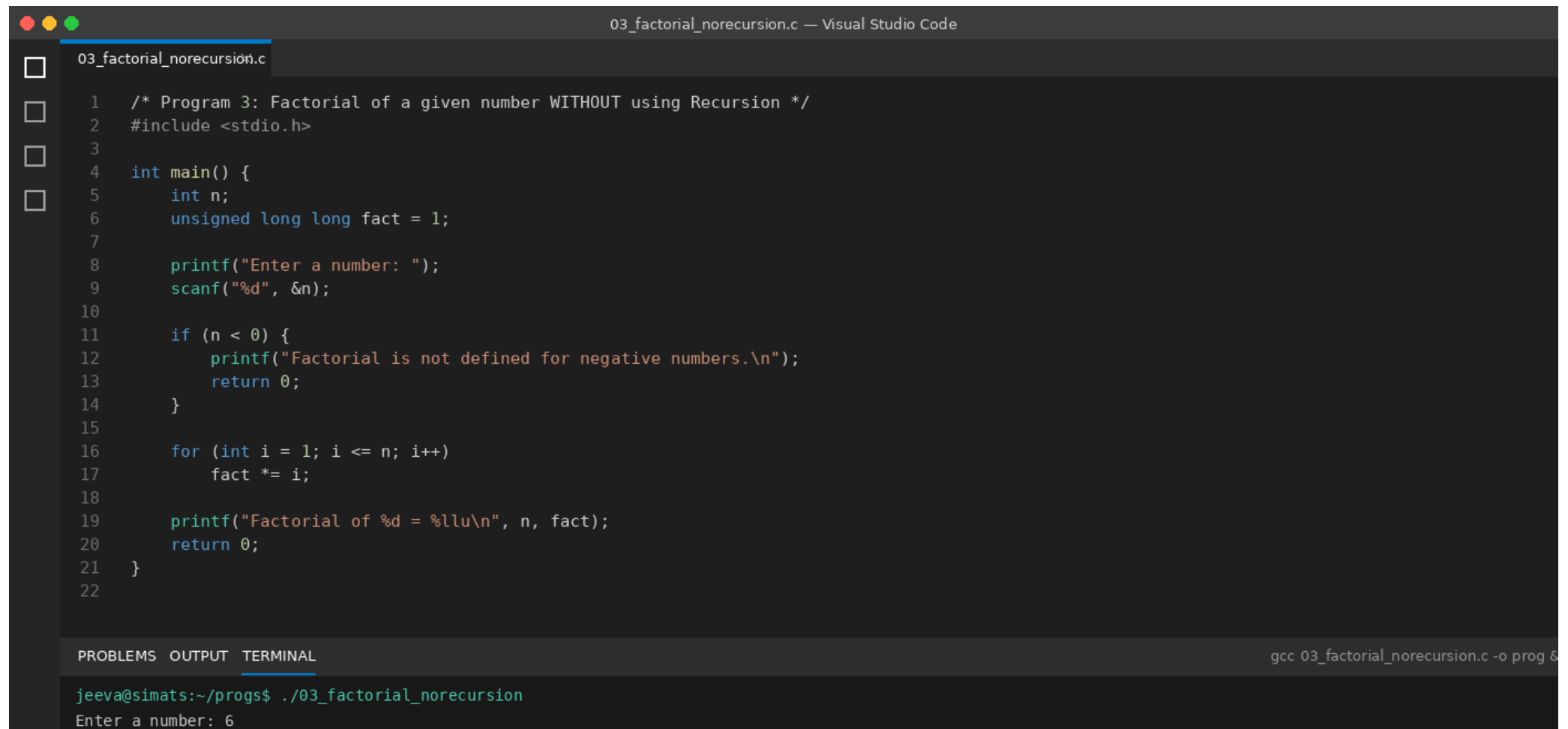
gcc 02_odd_even.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./02_odd_even
Enter number of elements: 5
Enter 5 numbers: 10 15 8 3 22

Result:
10 is Even
15 is Odd
8 is Even
3 is Odd

```



```
03_factorial_norecursion.c — Visual Studio Code

03_factorial_norecursion.c
1  /* Program 3: Factorial of a given number WITHOUT using Recursion */
2  #include <stdio.h>
3
4  int main() {
5      int n;
6      unsigned long long fact = 1;
7
8      printf("Enter a number: ");
9      scanf("%d", &n);
10
11     if (n < 0) {
12         printf("Factorial is not defined for negative numbers.\n");
13         return 0;
14     }
15
16     for (int i = 1; i <= n; i++)
17         fact *= i;
18
19     printf("Factorial of %d = %llu\n", n, fact);
20     return 0;
21 }
22

PROBLEMS OUTPUT TERMINAL
jeeva@simats:~/progs$ ./03_factorial_norecursion
Enter a number: 6
```

04_fibonacci_norecursion.c — Visual Studio Code

04_fibonacci_norecursion.c

```

1  /* Program 4: Fibonacci series WITHOUT using Recursion */
2  #include <stdio.h>
3
4  int main() {
5      int n;
6      printf("Enter number of terms: ");
7      scanf("%d", &n);
8
9      long long t1 = 0, t2 = 1, next;
10
11     printf("Fibonacci Series: ");
12     for (int i = 1; i <= n; i++) {
13         if (i == 1) { printf("%lld ", t1); continue; }
14         if (i == 2) { printf("%lld ", t2); continue; }
15         next = t1 + t2;
16         t1 = t2;
17         t2 = next;
18         printf("%lld ", next);
19     }
20     printf("\n");
21     return 0;
22 }
23

```

PROBLEMS OUTPUT TERMINAL

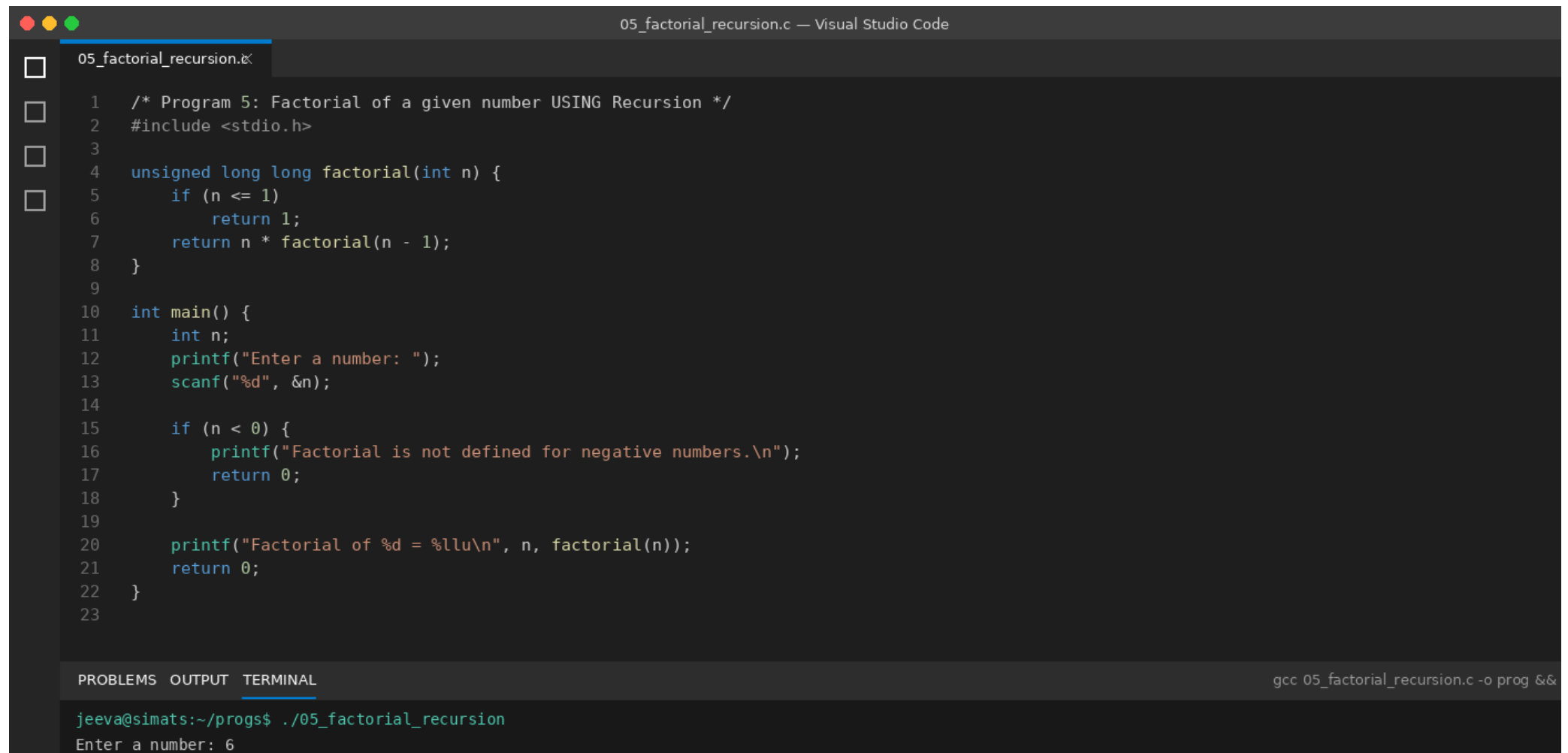
gcc 04_fibonacci_norecursion.c -o prog 4

```

jeeva@simats:~/progs$ ./04_fibonacci_norecursion
Enter number of terms: 8

```

Program 5: Factorial of a Number using Recursion



```
05_factorial_recursion.c — Visual Studio Code

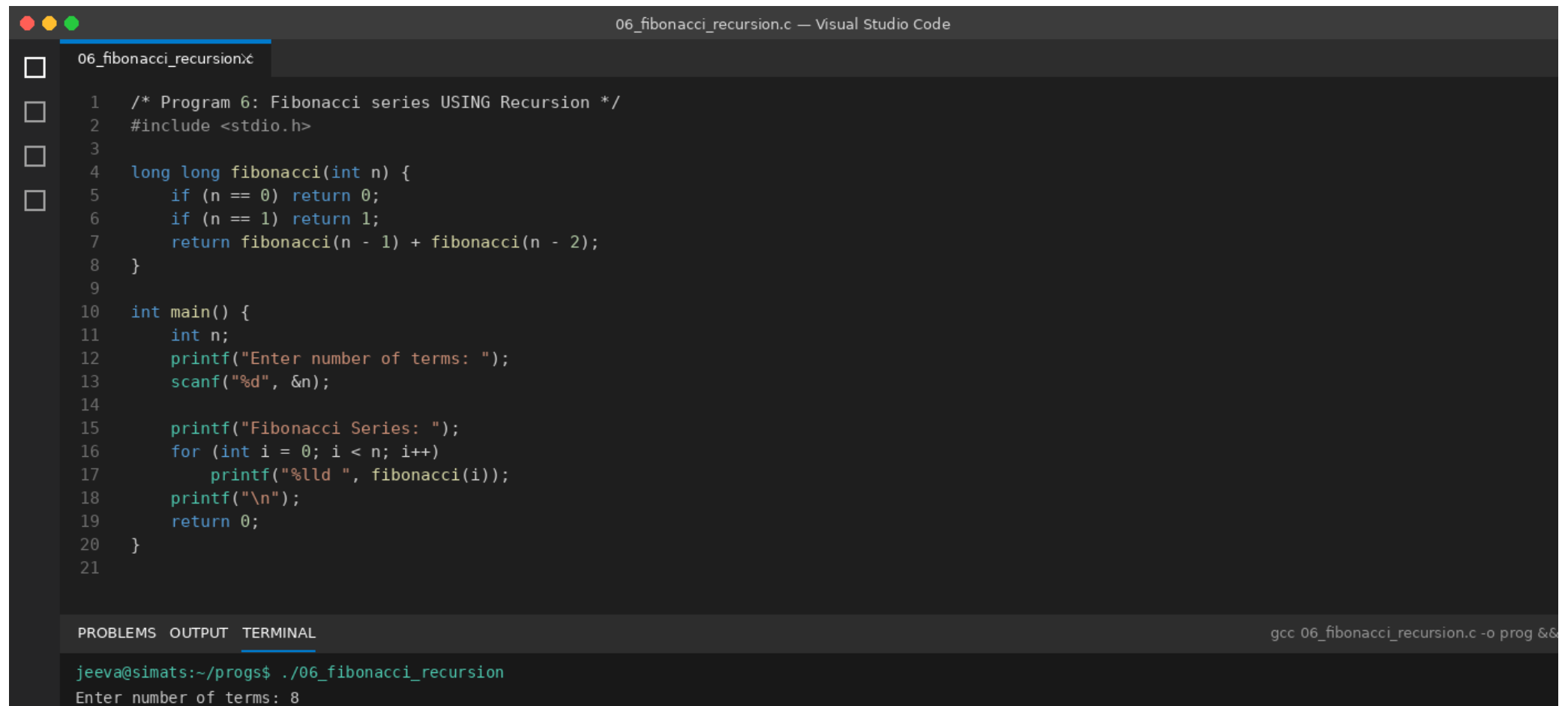
05_factorial_recursion.c
1  /* Program 5: Factorial of a given number USING Recursion */
2  #include <stdio.h>
3
4  unsigned long long factorial(int n) {
5      if (n <= 1)
6          return 1;
7      return n * factorial(n - 1);
8  }
9
10 int main() {
11     int n;
12     printf("Enter a number: ");
13     scanf("%d", &n);
14
15     if (n < 0) {
16         printf("Factorial is not defined for negative numbers.\n");
17         return 0;
18     }
19
20     printf("Factorial of %d = %llu\n", n, factorial(n));
21     return 0;
22 }
23
```

PROBLEMS OUTPUT TERMINAL

gcc 05_factorial_recursion.c -o prog &&

jeeva@simats:~/progs\$./05_factorial_recursion

Enter a number: 6



```
06_fibonacci_recursion.c — Visual Studio Code

06_fibonacci_recursion.c
1  /* Program 6: Fibonacci series USING Recursion */
2  #include <stdio.h>
3
4  long long fibonacci(int n) {
5      if (n == 0) return 0;
6      if (n == 1) return 1;
7      return fibonacci(n - 1) + fibonacci(n - 2);
8  }
9
10 int main() {
11     int n;
12     printf("Enter number of terms: ");
13     scanf("%d", &n);
14
15     printf("Fibonacci Series: ");
16     for (int i = 0; i < n; i++)
17         printf("%lld ", fibonacci(i));
18     printf("\n");
19     return 0;
20 }
21
```

PROBLEMS OUTPUT TERMINAL

jeewa@simats:~/progs\$./06_fibonacci_recursion
Enter number of terms: 8

gcc 06_fibonacci_recursion.c -o prog &&

07_array_ops.c

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

```

/* Program 7: Array operations - Insert, Delete and Display */
#include <stdio.h>

int arr[100], n = 0;

void display() {
    if (n == 0) { printf("Array is empty.\n"); return; }
    printf("Array elements: ");
    for (int i = 0; i < n; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

void insert() {
    int pos, val;
    printf("Enter position (0 to %d) to insert: ", n);
    scanf("%d", &pos);
    printf("Enter value to insert: ");
    scanf("%d", &val);

    if (pos < 0 || pos > n) { printf("Invalid position.\n"); return; }

    for (int i = n; i > pos; i--)
        arr[i] = arr[i - 1];
    arr[pos] = val;
    n++;
    ... (truncated for display) ...
}

```

PROBLEMS

OUTPUT

TERMINAL

gcc 07_array_ops.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./07_array_ops

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 1
Enter position (0 to 0) to insert: 0
Enter value to insert: 10
Inserted 10 at position 0.

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 1
Enter position (0 to 1) to insert: 1
Enter value to insert: 20
Inserted 20 at position 1.

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 3
Array elements: 10 20

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 2
Enter position (0 to 1) to delete: 0
Deleted element at position 0.

```

08_linear_search.c — Visual Studio Code

08_linear_search.c

```

1  /* Program 8: Search a number using Linear Search method */
2  #include <stdio.h>
3
4  int main() {
5      int n, key, found = 0;
6
7      printf("Enter number of elements: ");
8      scanf("%d", &n);
9
10     int arr[100];
11     printf("Enter %d elements: ", n);
12     for (int i = 0; i < n; i++)
13         scanf("%d", &arr[i]);
14
15     printf("Enter number to search: ");
16     scanf("%d", &key);
17
18     for (int i = 0; i < n; i++) {
19         if (arr[i] == key) {
20             printf("Element %d found at position %d.\n", key, i);
21             found = 1;
22             break;
23         }
24     }
25
26     if (!found)
27         ... (truncated for display) ...

```

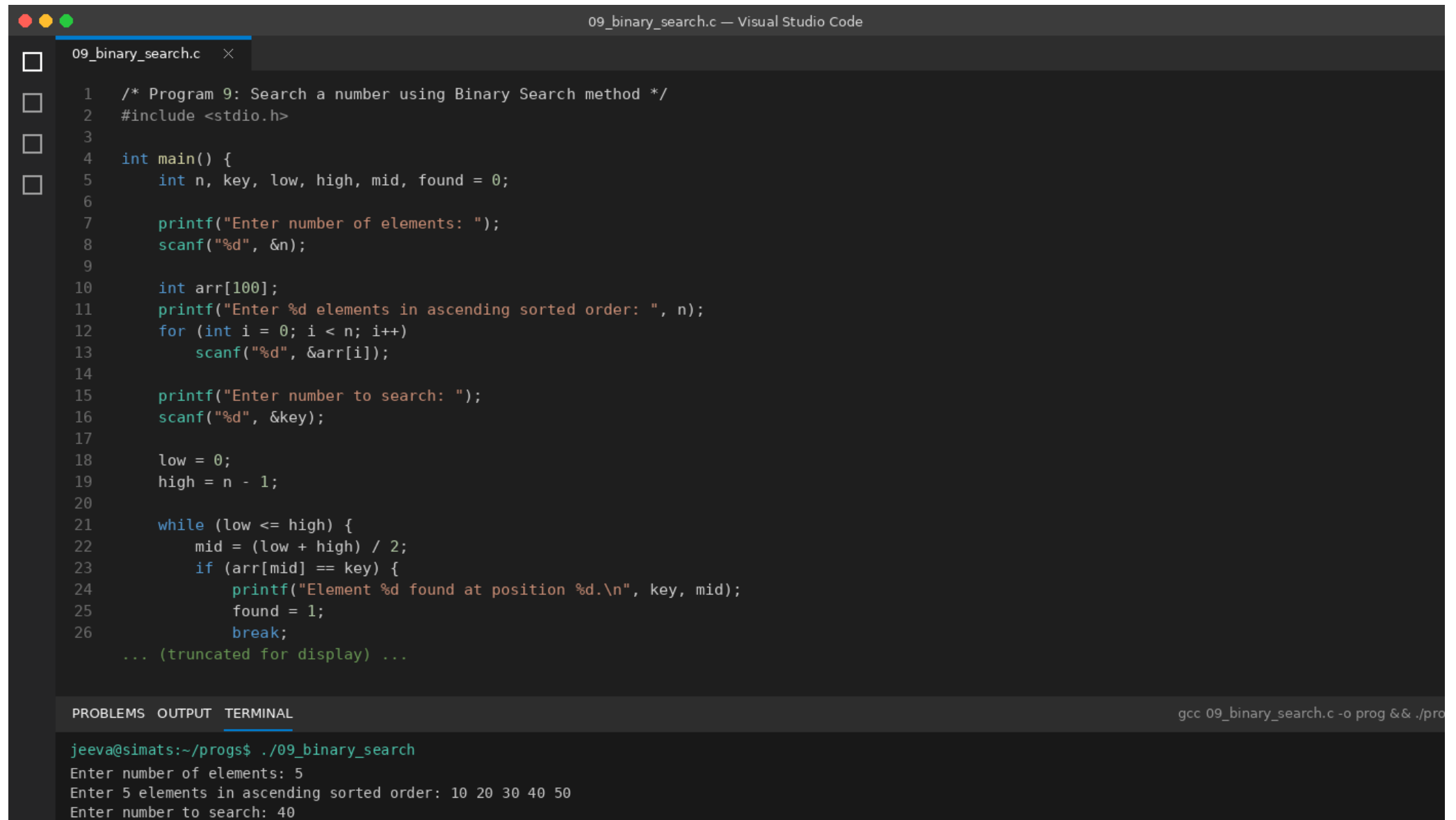
PROBLEMS OUTPUT TERMINAL

```

gcc 08_linear_search.c -o prog && ./prog

jeeva@simats:~/progs$ ./08_linear_search
Enter number of elements: 5
Enter 5 elements: 10 20 30 40 50
Enter number to search: 30

```



```
09_binary_search.c — Visual Studio Code

09_binary_search.c x
1  /* Program 9: Search a number using Binary Search method */
2  #include <stdio.h>
3
4  int main() {
5      int n, key, low, high, mid, found = 0;
6
7      printf("Enter number of elements: ");
8      scanf("%d", &n);
9
10     int arr[100];
11     printf("Enter %d elements in ascending sorted order: ", n);
12     for (int i = 0; i < n; i++)
13         scanf("%d", &arr[i]);
14
15     printf("Enter number to search: ");
16     scanf("%d", &key);
17
18     low = 0;
19     high = n - 1;
20
21     while (low <= high) {
22         mid = (low + high) / 2;
23         if (arr[mid] == key) {
24             printf("Element %d found at position %d.\n", key, mid);
25             found = 1;
26             break;
27         }
28     }
29     ... (truncated for display) ...
30 }
```

PROBLEMS OUTPUT TERMINAL

gcc 09_binary_search.c -o prog && ./prog

jeeva@simats:~/progs\$./09_binary_search

Enter number of elements: 5

Enter 5 elements in ascending sorted order: 10 20 30 40 50

Enter number to search: 40

The image shows a Visual Studio Code editor window titled "10_linked_list.c — Visual Studio Code". The editor displays a C program for linked list operations. The code includes headers for `<stdio.h>` and `<stdlib.h>`, defines a `Node` struct with `data` and `next` pointers, and implements functions for inserting nodes at the end and deleting nodes. The terminal output shows the program being executed, displaying a menu with options 1 (Insert), 2 (Delete), 3 (Display), and 0 (Exit). The user interacts with the program, inserting values 10, 20, and 30, and then deleting the value 20. The final output shows the linked list as "10 -> 20 -> 30 -> NULL" after the deletion of 20.

```
1  /* Program 10: Linked list operations */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  struct Node {
6      int data;
7      struct Node *next;
8  };
9
10 struct Node *head = NULL;
11
12 void insertEnd(int val) {
13     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
14     newNode->data = val;
15     newNode->next = NULL;
16
17     if (head == NULL) { head = newNode; return; }
18
19     struct Node *temp = head;
20     while (temp->next != NULL)
21         temp = temp->next;
22     temp->next = newNode;
23 }
24
25 void deleteVal(int val) {
26     struct Node *temp = head, *prev = NULL;
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL gcc 10_linked_list.c -o prog && ./prog

```
jeeva@simats:~/progs$ ./10_linked_list

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 1
Enter value to insert: 10

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 1
Enter value to insert: 20

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 1
Enter value to insert: 30

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 3
Linked List: 10 -> 20 -> 30 -> NULL

1.Insert 2.Delete 3.Display 0.Exit
Enter choice: 2
Enter value to delete: 20
Deleted 20

```

11_stack_array.c — Visual Studio Code

11_stack_array.c

```

1  /* Program 11: Stack operations such as PUSH, POP and PEEK */
2  #include <stdio.h>
3  #define MAX 100
4
5  int stack[MAX], top = -1;
6
7  void push(int val) {
8      if (top == MAX - 1) { printf("Stack Overflow.\n"); return; }
9      stack[++top] = val;
10     printf("Pushed %d\n", val);
11 }
12
13 void pop() {
14     if (top == -1) { printf("Stack Underflow.\n"); return; }
15     printf("Popped %d\n", stack[top--]);
16 }
17
18 void peek() {
19     if (top == -1) { printf("Stack is empty.\n"); return; }
20     printf("Top element: %d\n", stack[top]);
21 }
22
23 int main() {
24     int choice, val;
25     do {
26         printf("\n1.Push 2.Pop 3.Peek 0.Exit\nEnter choice: ");
27         ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL gcc 11_stack_array.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./11_stack_array

1.Push 2.Pop 3.Peek 0.Exit
Enter choice: 1
Enter value to push: 10
Pushed 10

1.Push 2.Pop 3.Peek 0.Exit
Enter choice: 1
Enter value to push: 20
Pushed 20

1.Push 2.Pop 3.Peek 0.Exit
Enter choice: 1
Enter value to push: 30
Pushed 30

1.Push 2.Pop 3.Peek 0.Exit
Enter choice: 3
Top element: 30

1.Push 2.Pop 3.Peek 0.Exit
Enter choice: 2

```

12_stack_infix_postfix.c — Visual Studio Code

12_stack_infix_postfix.c

```

1  /* Program 12: Application of Stack - Infix to Postfix Notation Conversion */
2  #include <stdio.h>
3  #include <string.h>
4  #include <ctype.h>
5
6  char stack[100];
7  int top = -1;
8
9  void push(char c) { stack[++top] = c; }
10 char pop() { return stack[top--]; }
11 char peekTop() { return stack[top]; }
12
13 int precedence(char c) {
14     if (c == '+' || c == '-') return 1;
15     if (c == '*' || c == '/') return 2;
16     if (c == '^') return 3;
17     return 0;
18 }
19
20 int main() {
21     char infix[100], postfix[100];
22     int i = 0, j = 0;
23
24     printf("Enter infix expression: ");
25     scanf("%s", infix);
26
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 12_stack_infix_postfix.c -o prog &&

```

jeeva@simats:~/progs$ ./12_stack_infix_postfix
Enter infix expression: a+b*(c-d)

```

13_queue_array.c — Visual Studio Code

13_queue_array.c

```

1  /* Program 13: Queue operations such as ENQUEUE, DEQUEUE and Display */
2  #include <stdio.h>
3  #define MAX 100
4
5  int queue[MAX], front = -1, rear = -1;
6
7  void enqueue(int val) {
8      if (rear == MAX - 1) { printf("Queue Overflow.\n"); return; }
9      if (front == -1) front = 0;
10     queue[++rear] = val;
11     printf("Enqueued %d\n", val);
12 }
13
14 void dequeue() {
15     if (front == -1 || front > rear) { printf("Queue Underflow.\n"); return; }
16     printf("Dequeued %d\n", queue[front++]);
17 }
18
19 void display() {
20     if (front == -1 || front > rear) { printf("Queue is empty.\n"); return; }
21     printf("Queue elements: ");
22     for (int i = front; i <= rear; i++)
23         printf("%d ", queue[i]);
24     printf("\n");
25 }
26
... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 13_queue_array.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./13_queue_array

1.Enqueue 2.Dequeue 3.Display 0.Exit
Enter choice: 1
Enter value to enqueue: 10
Enqueued 10

1.Enqueue 2.Dequeue 3.Display 0.Exit
Enter choice: 1
Enter value to enqueue: 20
Enqueued 20

1.Enqueue 2.Dequeue 3.Display 0.Exit
Enter choice: 1
Enter value to enqueue: 30
Enqueued 30

1.Enqueue 2.Dequeue 3.Display 0.Exit
Enter choice: 3
Queue elements: 10 20 30

1.Enqueue 2.Dequeue 3.Display 0.Exit
Enter choice: 2

```

14_tree_traversals.c

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

18

19

20

21

22

23

24

25

26

```

/* Program 14: Tree Traversals (Inorder, Preorder, Postorder) */
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *left, *right;
};

struct Node* newNode(int val) {
    struct Node* node = (struct Node*)malloc(sizeof(struct Node));
    node->data = val;
    node->left = node->right = NULL;
    return node;
}

struct Node* insert(struct Node* root, int val) {
    if (root == NULL) return newNode(val);
    if (val < root->data)
        root->left = insert(root->left, val);
    else
        root->right = insert(root->right, val);
    return root;
}

void inorder(struct Node* root) {
    ... (truncated for display) ...

```

PROBLEMS

OUTPUT

TERMINAL

```

gcc 14_tree_traversals.c -o prog && ./prog
jeeva@simats:~/progs$ ./14_tree_traversals
Enter number of nodes: 5
Enter 5 values to build the binary search tree: 50 30 70 20 40

Inorder Traversal:  20 30 40 50 70
Preorder Traversal: 50 30 20 40 70

```

15_hashing_linear_probing.c — Visual Studio Code

15_hashing_linear_probing.c

```

1  /* Program 15: Hashing using Linear Probing method */
2  #include <stdio.h>
3  #define SIZE 10
4
5  int hashTable[SIZE];
6
7  void initTable() {
8      for (int i = 0; i < SIZE; i++)
9          hashTable[i] = -1;
10 }
11
12 int hashFunc(int key) { return key % SIZE; }
13
14 void insert(int key) {
15     int idx = hashFunc(key);
16     int start = idx;
17
18     while (hashTable[idx] != -1) {
19         idx = (idx + 1) % SIZE;
20         if (idx == start) { printf("Hash table is full. Cannot insert %d.\n", key); return; }
21     }
22     hashTable[idx] = key;
23     printf("Inserted %d at index %d\n", key, idx);
24 }
25
26 void display() {
27     ... (truncated for display) ...
28 }

```

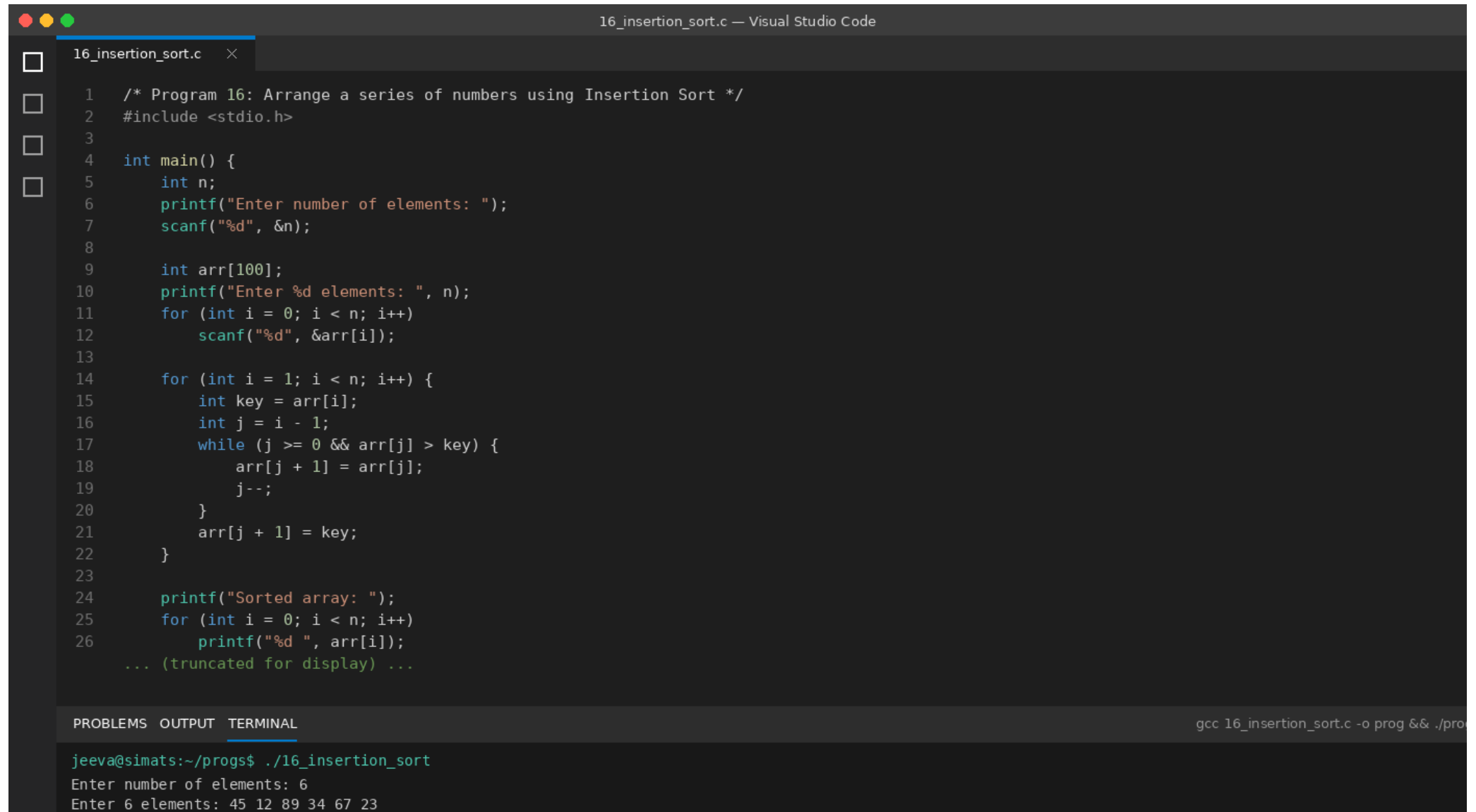
PROBLEMS OUTPUT TERMINAL gcc 15_hashing_linear_probing.c -o prog

```

jeeva@simats:~/progs$ ./15_hashing_linear_probing
Enter number of keys to insert: 5
Enter 5 keys: 15 25 35 5 45
Inserted 15 at index 5
Inserted 25 at index 6
Inserted 35 at index 7
Inserted 5 at index 8
Inserted 45 at index 9

Hash Table:
Index 0: -
Index 1: -
Index 2: -
Index 3: -
Index 4: -
Index 5: 15
Index 6: 25
Index 7: 35
Index 8: 5

```



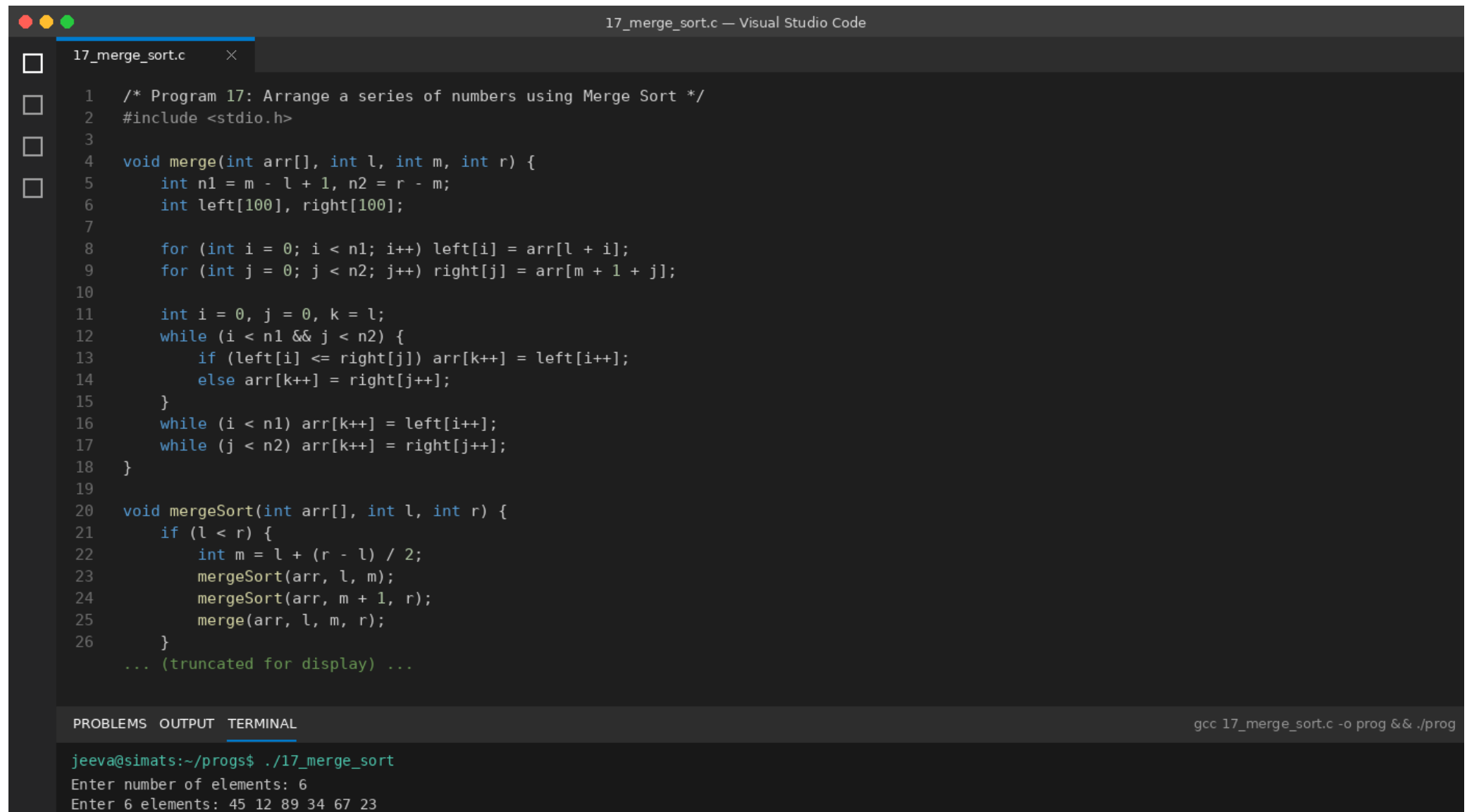
```
16_insertion_sort.c — Visual Studio Code

16_insertion_sort.c ×
1  /* Program 16: Arrange a series of numbers using Insertion Sort */
2  #include <stdio.h>
3
4  int main() {
5      int n;
6      printf("Enter number of elements: ");
7      scanf("%d", &n);
8
9      int arr[100];
10     printf("Enter %d elements: ", n);
11     for (int i = 0; i < n; i++)
12         scanf("%d", &arr[i]);
13
14     for (int i = 1; i < n; i++) {
15         int key = arr[i];
16         int j = i - 1;
17         while (j >= 0 && arr[j] > key) {
18             arr[j + 1] = arr[j];
19             j--;
20         }
21         arr[j + 1] = key;
22     }
23
24     printf("Sorted array: ");
25     for (int i = 0; i < n; i++)
26         printf("%d ", arr[i]);
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

```
gcc 16_insertion_sort.c -o prog && ./prog
jeeva@simats:~/progs$ ./16_insertion_sort
Enter number of elements: 6
Enter 6 elements: 45 12 89 34 67 23
```



```
17_merge_sort.c — Visual Studio Code

17_merge_sort.c ×
1  /* Program 17: Arrange a series of numbers using Merge Sort */
2  #include <stdio.h>
3
4  void merge(int arr[], int l, int m, int r) {
5      int n1 = m - l + 1, n2 = r - m;
6      int left[100], right[100];
7
8      for (int i = 0; i < n1; i++) left[i] = arr[l + i];
9      for (int j = 0; j < n2; j++) right[j] = arr[m + 1 + j];
10
11     int i = 0, j = 0, k = l;
12     while (i < n1 && j < n2) {
13         if (left[i] <= right[j]) arr[k++] = left[i++];
14         else arr[k++] = right[j++];
15     }
16     while (i < n1) arr[k++] = left[i++];
17     while (j < n2) arr[k++] = right[j++];
18 }
19
20 void mergeSort(int arr[], int l, int r) {
21     if (l < r) {
22         int m = l + (r - l) / 2;
23         mergeSort(arr, l, m);
24         mergeSort(arr, m + 1, r);
25         merge(arr, l, m, r);
26     }
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 17_merge_sort.c -o prog && ./prog

```
jeeva@simats:~/progs$ ./17_merge_sort
Enter number of elements: 6
Enter 6 elements: 45 12 89 34 67 23
```

Program 18: Quick Sort

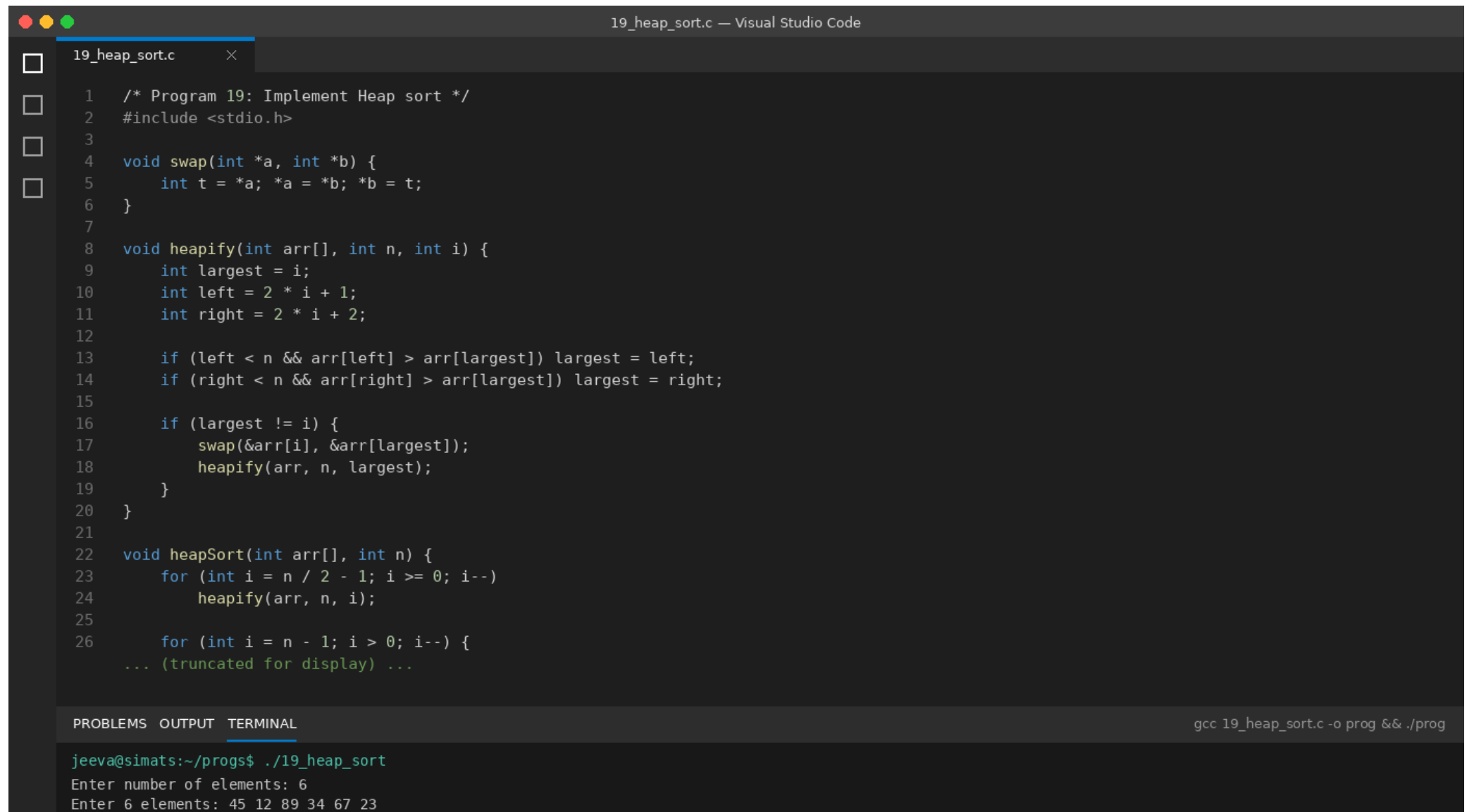
18_quick_sort.c — Visual Studio Code

```
18_quick_sort.c x
1  /* Program 18: Arrange a series of numbers using Quick Sort */
2  #include <stdio.h>
3
4  void swap(int *a, int *b) {
5      int t = *a; *a = *b; *b = t;
6  }
7
8  int partition(int arr[], int low, int high) {
9      int pivot = arr[high];
10     int i = low - 1;
11
12     for (int j = low; j < high; j++) {
13         if (arr[j] < pivot) {
14             i++;
15             swap(&arr[i], &arr[j]);
16         }
17     }
18     swap(&arr[i + 1], &arr[high]);
19     return i + 1;
20 }
21
22 void quickSort(int arr[], int low, int high) {
23     if (low < high) {
24         int pi = partition(arr, low, high);
25         quickSort(arr, low, pi - 1);
26         quickSort(arr, pi + 1, high);
27     }
28     ... (truncated for display) ...
29 }
```

PROBLEMS OUTPUT TERMINAL

gcc 18_quick_sort.c -o prog && ./prog

```
jeeva@simats:~/progs$ ./18_quick_sort
Enter number of elements: 6
Enter 6 elements: 45 12 89 34 67 23
```



```
19_heap_sort.c — Visual Studio Code

19_heap_sort.c ×
1  /* Program 19: Implement Heap sort */
2  #include <stdio.h>
3
4  void swap(int *a, int *b) {
5      int t = *a; *a = *b; *b = t;
6  }
7
8  void heapify(int arr[], int n, int i) {
9      int largest = i;
10     int left = 2 * i + 1;
11     int right = 2 * i + 2;
12
13     if (left < n && arr[left] > arr[largest]) largest = left;
14     if (right < n && arr[right] > arr[largest]) largest = right;
15
16     if (largest != i) {
17         swap(&arr[i], &arr[largest]);
18         heapify(arr, n, largest);
19     }
20 }
21
22 void heapSort(int arr[], int n) {
23     for (int i = n / 2 - 1; i >= 0; i--)
24         heapify(arr, n, i);
25
26     for (int i = n - 1; i > 0; i--) {
27         ... (truncated for display) ...
28     }
29 }

```

PROBLEMS OUTPUT TERMINAL

gcc 19_heap_sort.c -o prog && ./prog

jeeva@simats:~/progs\$./19_heap_sort

Enter number of elements: 6

Enter 6 elements: 45 12 89 34 67 23

20_avl_tree.c — Visual Studio Code

20_avl_tree.c

```

1  /* Program 20: AVL Tree - Insert, Delete, Search */
2  #include <stdio.h>
3  #include <stdlib.h>
4
5  struct Node {
6      int data, height;
7      struct Node *left, *right;
8  };
9
10 int height(struct Node *n) { return n == NULL ? 0 : n->height; }
11 int max(int a, int b) { return (a > b) ? a : b; }
12
13 struct Node* newNode(int val) {
14     struct Node* node = (struct Node*)malloc(sizeof(struct Node));
15     node->data = val;
16     node->left = node->right = NULL;
17     node->height = 1;
18     return node;
19 }
20
21 struct Node* rightRotate(struct Node* y) {
22     struct Node* x = y->left;
23     struct Node* T2 = x->right;
24     x->right = y;
25     y->left = T2;
26     y->height = max(height(y->left), height(y->right)) + 1;
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL gcc 20_avl_tree.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./20_avl_tree

1.Insert 2.Delete 3.Search 4.Display(Inorder) 0.Exit
Enter choice: 1
Enter value to insert: 30
Inserted 30

1.Insert 2.Delete 3.Search 4.Display(Inorder) 0.Exit
Enter choice: 1
Enter value to insert: 20
Inserted 20

1.Insert 2.Delete 3.Search 4.Display(Inorder) 0.Exit
Enter choice: 1
Enter value to insert: 40
Inserted 40

1.Insert 2.Delete 3.Search 4.Display(Inorder) 0.Exit
Enter choice: 1
Enter value to insert: 10
Inserted 10

1.Insert 2.Delete 3.Search 4.Display(Inorder) 0.Exit

```

21_bfs.c — Visual Studio Code

21_bfs.c

```

1  /*
2   * Program 21: Write a C program to Graph traversal using Breadth First Search
3   */
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  #define MAX 20
8
9  int adj[MAX][MAX];
10 int visited[MAX];
11 int n;
12
13 void bfs(int start) {
14     int queue[MAX], front = 0, rear = 0;
15     visited[start] = 1;
16     queue[rear++] = start;
17
18     printf("BFS Traversal starting from vertex %d:\n", start);
19
20     while (front < rear) {
21         int current = queue[front++];
22         printf("%d ", current);
23
24         for (int i = 0; i < n; i++) {
25             if (adj[current][i] == 1 && !visited[i]) {
26                 visited[i] = 1;
27                 queue[rear++] = i;
28             }
29         }
30     }
31     printf("\n");
32 }
33
34 int main() {
35     int u, v;
36     printf("Enter number of vertices: ");
37     scanf("%d", &n);
38     printf("Enter number of edges: ");
39     scanf("%d", &e);
40     printf("Enter edges (u v) with 0-based vertex numbers:\n");
41     for (int i = 0; i < e; i++) {
42         scanf("%d %d", &u, &v);
43         adj[u][v] = 1;
44         adj[v][u] = 1;
45     }
46     printf("Enter starting vertex for BFS: ");
47     scanf("%d", &start);
48     bfs(start);
49     return 0;
50 }

```

PROBLEMS OUTPUT TERMINAL

gcc 21_bfs.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./21_bfs
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v) with 0-based vertex numbers:
0 1
0 2
1 3
2 4
3 5
4 5
2 5
Enter starting vertex for BFS: 0
BFS Traversal starting from vertex 0:
0 1 2 3 4 5

```

22_dfs.c — Visual Studio Code

22_dfs.c

```

1  /*
2   * Program 22: Write a C program to Graph traversal using Depth First Search
3   */
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  #define MAX 20
8
9  int adj[MAX][MAX];
10 int visited[MAX];
11 int n;
12
13 void dfs(int current) {
14     visited[current] = 1;
15     printf("%d ", current);
16
17     for (int i = 0; i < n; i++) {
18         if (adj[current][i] == 1 && !visited[i]) {
19             dfs(i);
20         }
21     }
22 }
23
24 int main() {
25     int edges, u, v, start;
26
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 22_dfs.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./22_dfs
Enter number of vertices: 6
Enter number of edges: 7
Enter edges (u v) with 0-based vertex numbers:
0 1
0 2
1 3
2 4
3 5
4 5
2 5
Enter starting vertex for DFS: 0
DFS Traversal starting from vertex 0:

```

23_dijkstra.c — Visual Studio Code

23_dijkstra.c

```

1  /*
2  * Program 23: Implementation of Shortest Path Algorithms using Dijkstra's Algorithm
3  */
4  #include <stdio.h>
5  #include <limits.h>
6
7  #define MAX 20
8
9  int graph[MAX][MAX];
10 int n;
11
12 int minDistance(int dist[], int visited[]) {
13     int min = INT_MAX, minIndex = -1;
14
15     for (int v = 0; v < n; v++) {
16         if (!visited[v] && dist[v] <= min) {
17             min = dist[v];
18             minIndex = v;
19         }
20     }
21     return minIndex;
22 }
23
24 void printSolution(int dist[], int src) {
25     printf("\nVertex \t Distance from Source (%d)\n", src);
26     for (int i = 0; i < n; i++)
27         ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 23_dijkstra.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./23_dijkstra
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (u v weight) with 0-based vertex numbers:
0 1 4
0 2 1
2 1 2
1 3 1
2 3 5
3 4 3
Enter source vertex: 0

Vertex    Distance from Source (0)
0         0
1         3
2         1
3         4

```

24_prims.c — Visual Studio Code

24_prims.c

```

1  /*
2  * Program 24: Implementation of Minimum Spanning Tree using Prim's Algorithm
3  */
4  #include <stdio.h>
5  #include <limits.h>
6
7  #define MAX 20
8
9  int graph[MAX][MAX];
10 int n;
11
12 int minKey(int key[], int mstSet[]) {
13     int min = INT_MAX, minIndex = -1;
14
15     for (int v = 0; v < n; v++) {
16         if (!mstSet[v] && key[v] < min) {
17             min = key[v];
18             minIndex = v;
19         }
20     }
21     return minIndex;
22 }
23
24 void printMST(int parent[]) {
25     int totalWeight = 0;
26     printf("\nEdge \tWeight\n");
27     ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 24_prims.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./24_prims
Enter number of vertices: 5
Enter number of edges: 6
Enter edges (u v weight) with 0-based vertex numbers:
0 1 2
0 3 6
1 2 3
1 3 8
1 4 5
3 4 9

Edge    Weight
0 - 1   2
1 - 2   3
0 - 3   6
1 - 4   5

```

25_kruskal.c — Visual Studio Code

25_kruskal.c

```

1  /*
2   * Program 25: Implementation of Minimum Spanning Tree using Kruskal's Algorithm
3   */
4  #include <stdio.h>
5  #include <stdlib.h>
6
7  #define MAX 50
8
9  typedef struct {
10     int u, v, weight;
11 } Edge;
12
13 int parent[MAX];
14
15 int find(int i) {
16     while (parent[i] != i)
17         i = parent[i];
18     return i;
19 }
20
21 void unionSet(int i, int j) {
22     int a = find(i);
23     int b = find(j);
24     parent[a] = b;
25 }
26
27 ... (truncated for display) ...

```

PROBLEMS OUTPUT TERMINAL

gcc 25_kruskal.c -o prog && ./prog

```

jeeva@simats:~/progs$ ./25_kruskal
Enter number of vertices: 4
Enter number of edges: 5
Enter edges (u v weight) with 0-based vertex numbers:
0 1 10
0 2 6
0 3 5
1 3 15
2 3 4

Edges in Minimum Spanning Tree:
2 - 3 4
0 - 3 5
0 - 1 10

```